

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E07218: 献给阿尔吉侬的花束

bfs, <http://cs101.openjudge.cn/practice/07218/>

思路：

一道相对简单的BFS题目。使用队列来存储将要走的位置的index，将矩阵对应位置设置为已走过，并用popleft依次遍历即可。

代码：

```
from collections import deque

def dfs(matr, start, r, c):
    direct = ((-1, 0), (1, 0), (0, 1), (0, -1))
    q = deque([start])
    while q:
        x0, y0, step = q.popleft()
        for i in direct:
            x1 = x0 + i[0]
            y1 = y0 + i[1]
```

```

        if x1 < 0 or y1 < 0 or x1 > r - 1 or y1 > c - 1:
            continue
        if matr[x1][y1] == 'E':
            return step + 1
        elif matr[x1][y1] == '.':
            matr[x1][y1] = '#'
            q.append((x1, y1, step + 1))

```

```

def main():
    n = int(input())
    for i in range(n):
        r, c = [int(x) for x in input().split()]
        matr = []
        for j in range(r):
            matr.append(list(input()))
        start = (0, 0, 0)
        for x in range(r):
            for y in range(c):
                if matr[x][y] == 'S':
                    start = (x, y, 0)
        result = dfs(matr, start, r, c)
        if result is not None:
            print(result)
        else:
            print('oop!')

```

```

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50887415提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import deque

def dfs(matr, start, r, c):
    direct = ((-1, 0), (1, 0), (0, 1), (0, -1))
    q = deque([start])
    while q:
        x0, y0, step = q.popleft()
        for i in direct:
            x1 = x0 + i[0]
            y1 = y0 + i[1]
            if x1 < 0 or y1 < 0 or x1 > r - 1 or y1 > c - 1:
                continue
            if matr[x1][y1] == 'E':
                return step + 1
            elif matr[x1][y1] == '.':
                matr[x1][y1] = '#'
                q.append((x1, y1, step + 1))

def main():
    n = int(input())
    for i in range(n):
        r, c = [int(x) for x in input().split()]
        matr = []
```

基本信息

#: 50887415
题目: 07218
提交人: 22n2200011816(略约横溪)
内存: 3988kB
时间: 80ms
语言: Python3
提交时间: 2025-11-18 11:01:04

M27925: 小组队列

dict, queue, <http://cs101.openjudge.cn/practice/27925/>

思路:

这道题的难点在于如何快速地判断队列中是否有自己队伍的人，并将其添加到对应队列中。

为此，我使用一个列表，如果有队伍*i*的人加入队列，就在index=*i*的位置+1，反正就-1，这样，如果后续有同样是队列*i*的人加入，只要读取index=*i*的位置即可判断；再使用一个字典teams_index，以某个编号的人为键，所属队伍为值进行存储。

随后就是进行ENQUEUE和DEQUEUE的操作。在这里，我使用deque套deque的方式，进行大的队列存储和队伍在队列中的具体为此存储。具体而言，对ENQUEUE，首先读取teams_index是否在teams_index的键中，是，则判断列表的index是否为0，若为0，说明队伍中没有人 在队列中，将[队伍index, deque(编号)]添加入队列，否则，就检索队列中的列表，当index相等时，就添加到对应的deque中；对DEQUEUE，则将队列中的第一个列表内的deque进行popleft操作，如果操作后队列为空，就对大的队列进行popleft即可。

代码:

```
from collections import deque

def main():
    n = int(input())
    teams = [0]*n # 用于存储小队中处于队列的人数
    teams_index = dict() # 用于存储小队队员属于哪个队列
```

```

for i in range(n):
    for j in [int(x) for x in input().split()]:
        teams_index[j] = i
q = deque()
while 1:
    os = input().split()
    if os[0] == 'STOP':
        break
    elif os[0] == 'ENQUEUE':
        identifier = int(os[1])
        if identifier in teams_index.keys():
            index = teams_index[identifier]
            if teams[index] == 0:
                q.append([index, deque([identifier])])
            else:
                for i in q:
                    if i[0] == index:
                        i[1].append(identifier)
                teams[index] += 1
        else:
            q.append([None, deque([identifier])])
    else:
        print(q[0][1].popleft())
        if q[0][0] is not None:
            teams[q[0][0]] -= 1
        if not q[0][1]:
            q.popleft()

if __name__ == '__main__':
    main()

```

代码运行截图（至少包含有"Accepted"）

#50887909提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
from collections import deque

def main():
    n = int(input())
    teams = [0]*n # 用于存储小队中处于队列的人数
    teams_index = dict() # 用于存储小队队员属于哪个队列
    for i in range(n):
        for j in [int(x) for x in input().split()]:
            teams_index[j] = i
    q = deque()
    while 1:
        os = input().split()
        if os[0] == 'STOP':
            break
        elif os[0] == 'ENQUEUE':
            identifier = int(os[1])
            if identifier in teams_index.keys():
                index = teams_index[identifier]
                if teams[index] == 0:
                    q.append([index, deque([identifier])])
            else:
                for i in q:
                    if i[0] == index:
```

基本信息

#: 50887909
题目: 27925
提交人: 22n2200011816(略约横溪)
内存: 4908kB
时间: 101ms
语言: Python3
提交时间: 2025-11-18 11:56:14

M04089: 电话号码

trie, <http://cs101.openjudge.cn/practice/04089/>

思路:

这道题目按照字典树的方法解决。将电话号码由短到长排列，先将最短的电话号码建成字典树（即上一个号码为key为下一个号码为值，依次嵌套），随后依次比较较长的号码：如果号码中的元素在树中遍历后可以完美符合较短的电话号码，则输出NO，否则，就在出现不同的位置嵌套创建键值对。

代码:

```
def build_tree(nums: list):
    trie = dict()
    for i in range(len(nums)):
        if i == 0:
            if len(nums) == 1:
                temp0 = 'NO'
            else:
                temp0 = dict()
                trie[nums[i]] = temp0
        elif i == len(nums)-1:
            temp0[nums[i]] = 'NO'
        else:
            temp1 = dict()
            temp0[nums[i]] = temp1
```

```

        temp0 = temp1
    return trie

def is_ok(tries: list, nums: list):
    for i in tries:
        temp_dict = i
        for j in range(len(nums)):
            if nums[j] in temp_dict.keys():
                temp_dict = temp_dict[nums[j]]
                if temp_dict == 'NO':
                    return 0
            elif j == len(nums) - 1:
                temp_dict[nums[j]] = 'NO'
            else:
                temp_dict1 = dict()
                temp_dict[nums[j]] = temp_dict1
                temp_dict = temp_dict1
    return tries

def main():
    n = int(input())
    for _ in range(n):
        t = int(input())
        temp = []
        for i in range(t):
            temp.append([int(x) for x in list(input())])
        temp.sort(key=len)
        tries = [build_tree(temp[0])]
        for i in range(1, len(temp)):
            temp1 = is_ok(tries, temp[i])
            if temp1 == 0:
                print('NO')
                break
            else:
                tries = temp1
        else:
            print('YES')

if __name__ == '__main__':
    main()

```

代码运行截图 (至少包含有"Accepted")

#50891867提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
def build_tree(nums: list):
    trie = dict()
    for i in range(len(nums)):
        if i == 0:
            if len(nums) == 1:
                temp0 = 'NO'
            else:
                temp0 = dict()
                trie[nums[i]] = temp0
        elif i == len(nums)-1:
            temp0[nums[i]] = 'NO'
        else:
            temp1 = dict()
            temp0[nums[i]] = temp1
            temp0 = temp1
    return trie

def is_ok(tries: list, nums: list):
    for i in tries:
        temp_dict = i
        for j in range(len(nums)):
            if nums[j] in temp_dict.keys():
                temp_dict[nums[j]] = temp_dict[nums[j]]
            else:
                temp_dict[nums[j]] = temp_dict[nums[j]]
```

基本信息

#: 50891867
题目: 04089
提交人: 22n2200011816(略约横溪)
内存: 16576kB
时间: 232ms
语言: Python3
提交时间: 2025-11-18 16:10:57

M3532.针对图的路径存在性查询I

disjoint set, <https://leetcode.cn/problems/path-existence-queries-in-a-graph-i/>

思路:

看了题解,发现可以用dp的方法解,不过既然用了更加类似图的方式,也就保留代码了。具体而言,使用字典来存储图,依次比较相近i和i-1的值,如果差小于maxdiff,就说明这两个节点联通,先将节点i加入i-1的联通节点中,再将i-1的联通节点浅拷贝到i的联通节点中。随后,对查询两个节点x,y是否存在路径,只需要判断y是否在x的联通图内即可。

代码

```
from typing import List, Optional
from collections import deque

class Solution:

    def build_graph(self, n, nums, maxDiff: int):
        graph = dict()
        for i in list(range(n)):
            graph[i] = {i}
        for i in range(1, len(nums)):
            j = i - 1
            if abs(nums[i] - nums[j]) <= maxDiff:
                graph[j].add(i)
```

```

        graph[i] = graph[j]
    return graph

    def pathExistenceQueries(self, n: int, nums: List[int], maxDiff: int,
queries: List[List[int]]) -> List[bool]:
        graph = self.build_graph(n, nums, maxDiff)
        ans = []
        for i in queries:
            ans.append(i[1] in graph[i[0]])
        return ans

```

代码运行截图 (至少包含有"Accepted")

通过 550 / 550 个通过的测试用例

Happy l3osefod 提交于 2025.11.18 17:43

写题解

⌚ 执行用时分布

511 ms | 击败 6.29%

🌟 复杂度分析

💾 消耗内存分布

76.81 MB | 击败 5.03%



代码 | Python3

```

from typing import List, Optional
from collections import deque

class Solution:

    def build_graph(self, n, nums, maxDiff: int):
        graph = dict()

```

M19943: 图的拉普拉斯矩阵

OOP, graph, implementation, <http://cs101.openjudge.cn/pctbook/E19943/>

要求创建Graph, Vertex两个类，建图实现。

思路：

创建了Graph, Vertex两个类，其中Vertex存储了节点的值和临近的节点，图则利用字典，存储将节点的值作为键，节点本身作为值，方便仅利用节点的值获得neighbors的信息。最后按要求输出拉普拉斯矩阵即可——对角线的大小为len，而[vertex][neighbor] = -1。

代码

```
class Vertex:
    def __init__(self, index=None):
        self.index = index
        self.neighbors = set()

class Graph:
    def __init__(self):
        self.vertices = dict()

def build_graph(g: Graph, vert1: int, vert2: int):
    temp_vert = Vertex(vert1)
    if vert1 in g.vertices.keys():
        temp_vert = g.vertices[vert1]
    temp_vert.neighbors.add(vert2)
    g.vertices[vert1] = temp_vert

    temp_vert = Vertex(vert2)
    if vert2 in g.vertices.keys():
        temp_vert = g.vertices[vert2]
    temp_vert.neighbors.add(vert1)
    g.vertices[vert2] = temp_vert

    return g

def build_matr(n, g: Graph):
    matr = [[0 for _ in range(n)] for _ in range(n)]
    for i in g.vertices.keys():
        temp_vert = g.vertices[i]
        matr[i][i] = len(temp_vert.neighbors)
        for j in temp_vert.neighbors:
            matr[i][j] = -1
    return matr

def main():
    num, edges = [int(x) for x in input().split()]
    g0 = Graph()
    for i in range(edges):
```

```
    vert1, vert2 = [int(x) for x in input().split()]
    g0 = build_graph(g0, vert1, vert2)
    matr = build_matr(num, g0)
    for i in matr:
        print(' '.join([str(x) for x in i]))

if __name__ == '__main__':
    main()
```

代码运行截图（至少包含有"Accepted"）

#50895640提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
class Vertex:
    def __init__(self, index=None):
        self.index = index
        self.neighbors = set()

class Graph:
    def __init__(self):
        self.vertices = dict()

def build_graph(g: Graph, vert1: int, vert2: int):
    temp_vert = Vertex(vert1)
    if vert1 in g.vertices.keys():
        temp_vert = g.vertices[vert1]
    temp_vert.neighbors.add(vert2)
    g.vertices[vert1] = temp_vert

    temp_vert = Vertex(vert2)
    if vert2 in g.vertices.keys():
        temp_vert = g.vertices[vert2]
    temp_vert.neighbors.add(vert1)
    g.vertices[vert2] = temp_vert
```

基本信息

#: 50895640
题目: E19943
提交人: 22n2200011816(略约横溪)
内存: 3700kB
时间: 21ms
语言: Python3
提交时间: 2025-11-18 20:06:06

2. 学习总结和个人收获

虽然过了期中周，但是期中堆积的作业和DDL依然让人压力山大，甚至最后一题排队没有写出来（写了个超时的解法）。怀念期初可以无忧无虑写代码的日子。